

Ethereum SDK Audit (initial report, 2026-05-08)

Repository	https://gitlab.com/quip.network/ethereum-sdk
Commit	29421e2997ccc08879f0c5bdcfdbce27ccb2d4ab
Documentations	• Inline code comments • <code>README.md</code> • <code>DEPLOYMENTS.md</code> • <code>MIDL-DEPLOYMENT.md</code> • <code>MIDL-REOWN-INTEGRATION.md</code> • <code>TODO.md</code> • <code>lib/README.md</code> • <code>remappings.txt</code>
Tests	• TypeScript tests: ◦ <code>src/addresses.test.ts</code> ◦ <code>src/wotsCodec.test.ts</code> ◦ <code>test/midl-integration.ts</code> • Solidity test suites: ◦ <code>test/Deployer/*</code> ◦ <code>test/QuipFactory/*</code> ◦ <code>test/QuipPaymaster/*</code> ◦ <code>test/QuipWallet/*</code> ◦ <code>test/WOTSPlusCodec/*</code> ◦ <code>test/integration/*</code> ◦ <code>test/libraries/*</code> • Support files in the test tree: ◦ harnesses in <code>test/harness/*</code> ◦ fixtures in <code>test/fixtures/*</code> ◦ test-local package manifest: <code>test/package.json</code>



Audit Report Summary

- overall repo quality is high, and the existing test coverage is already above typical audit baselines

- 2 high-severity issues: under-bound paymaster sponsorship signatures and non-monotonic wallet WOTS+ key burning
- several medium-severity issues: replayable key-management signatures, self-authorized upgrade verification, and a stale TypeScript SDK / codec surface
- lower-severity issues: misleading vaultId namespacing, operational ERC-1271 replay resistance, and recovery flows that are less independent than implied

Aspect	Rating
Smart contract architecture	
Core security design	
Test coverage	
Documentation / operational clarity	
Build / release hygiene	
Deployment / packaging completeness	

Table Of Contents

1. Scope And System Overview
2. Primary Findings
 - a. Paymaster sponsorship signatures are under-bound to the actual UserOperation envelope
 - b. Wallet WOTS+ keys are not burned monotonically after first exposure
 - c. `addKeys` and `refreshKeys` share the same signed digest for recovery and verification keysets
 - d. Upgrade verification signatures are self-authorized rather than independently authorized
 - e. ERC-1271 replay resistance is operational rather than enforced on-chain

- f. The SDK still targets a stale wallet surface
- g. The factory still presents a misleading owner-scoped `vaultId` namespace

3. Operational And Repository Risks

- a. The repository mixes multiple contradictory build and release assumptions
- b. The SDK has additional safety and usability failures beyond simple stale methods
- c. The paymaster is in scope for security, but not in scope for the repo's deployment or packaging flows
- d. The package is not self-contained for clean SDK or Forge consumers
- e. Important safety properties are left as TODOs rather than enforced invariants
- f. Factory governance and rollout policy remain under-specified
- g. The assembly is generally deliberate, but a few maintenance hazards remain
- h. Repository documentation and CI overstate operational maturity

4. Testing Gaps

- a. Missing regression tests for paymaster envelope binding
- b. Missing stateful and invariant campaigns for wallet key lifecycle
- c. Missing regression and fuzz tests for `addKeys` vs `refreshKeys` misbinding
- d. Missing stateful and invariant campaigns for paymaster verifier lifecycle
- e. Missing higher-level invariant campaigns for factory implementation lifecycle
- f. Missing client-surface and parity tests for the published SDK
- g. Missing targeted tests for upgrade authorization semantics
- h. Missing explicit behavioral tests for ERC-1271 key lifecycle
- i. Missing explicit fuzz and invariant configuration in `foundry.toml`

5. Overall Assessment

Scope And System Overview

This repository contains four security-sensitive surfaces:

- **QuipWallet**
 - the main trust anchor; holds funds, rotates one-time WOTS+ keys, verifies upgrade payloads, exposes ERC-4337 execution, and provides recovery paths
- **QuipFactory**
 - governs vetted wallet implementations, deterministic deployment, and fee policy
- **QuipPaymaster**
 - authorizes sponsored ERC-4337 operations using per-wallet WOTS+ verifier chains
- the TypeScript SDK and build pipeline
 - constructs payloads, selects network addresses, and packages the client surface shipped to integrators

The core invariants are:

- WOTS+ keys must be one-time-use and never become live again after exposure
- signatures must be bound to the exact operation semantics they are authorizing
- replay protection must be applied before any external interaction
- factory rollout policy must be at least as strict as the wallet upgrade model assumes
- the SDK and packaging pipeline must remain aligned with the live contract surface

Primary Findings

1. Paymaster sponsorship signatures are under-bound to the actual UserOperation envelope

Severity: High

`QuipPaymaster` verifies a digest built only from:

- sender
- nonce
- `hashCalldata(callData)`
- current verifier
- next verifier
- chain ID
- paymaster address

It does not bind:

- `validUntil`
- `validAfter`
- `paymasterVerificationGasLimit`
- `paymasterPostOpGasLimit`
- `accountGasLimits`
- `preVerificationGas`
- `gasFees`
- `maxCost`
- `userOpHash`

Relevant references:

- `contracts/QuipPaymaster.sol:97`
 - `validatePaymasterUserOp(...)` ignores `userOpHash` and `maxCost`
- `contracts/QuipPaymaster.sol:135`
 - `validUntil` and `validAfter` are unpacked only after signature verification

- `contracts/QuipPaymaster.sol:364`
 - the digest is built without the broader gas and validity envelope
- `test/QuipPaymaster/QuipPaymaster.t.sol:125`
 - the helper test path explicitly documents the constituent-field digest shape

Impact:

- one backend-generated approval is not tied to one exact sponsorship package
- a valid sponsorship signature can be reused with different gas-budget or validity settings than the signer may have intended
- this weakens the paymaster's approval semantics even when the wallet-side authorization remains sound

Recommendation:

- bind the approval to the full intended sponsorship envelope, ideally `userOpHash` or an equivalent complete envelope hash
- add regression tests that mutate each excluded field and assert whether validation must fail



Attack scenario

1. A backend signs sponsorship for one wallet action under conservative gas and validity terms.
2. An attacker obtains the signed sponsored operation before inclusion.
3. The attacker keeps the signed fields unchanged but inflates unsigned gas and validity fields.
4. The paymaster still validates the signature and funds the same action under materially worse economic terms.

2. Wallet WOTS+ keys are not burned monotonically after first exposure

Severity: High

The wallet checks whether a key is currently installed, not whether it has ever been used before. A previously revealed WOTS+ key can therefore become live again if it is later reintroduced through a reset, migration, or reinitialization path.

Relevant references:

- `contracts/QuipWallet.sol:1181`
 - `_isKeyInUse(...)` checks live membership only
- `contracts/QuipWallet.sol:1205`
 - `_enforceUnusedKey(...)` inherits that non-monotonic notion of freshness
- `contracts/QuipWallet.sol:1236`
 - `_safeAddKey(...)` can therefore reintroduce a historically exposed key

Impact:

- replay resistance for a one-time signature scheme depends on permanent burn after exposure
- reinstalling a previously revealed key turns old signature material back into live authorization
- this is inconsistent with the paymaster verifier path, which already uses the stricter monotonic "once seen, never reused" rule

Recommendation:

- add a monotonic wallet-wide `usedKey` index keyed by `(publicSeed, publicKeyHash)` across all wallet key domains
- reject reinstallation of any key that has ever existed in the wallet



Attack scenario

1. A legitimate wallet action is executed under transaction key `A`, rotating to `B`, and the signed payload becomes public on-chain.

2. Later, through migration, reset, or reinitialization, key `A` is accidentally reintroduced as a live key.
3. An attacker replays the old signed payload that was already revealed on-chain.
4. The wallet accepts the replay because `A` is live again and there is no historical record that it was already used.

3. `addKeys` and `refreshKeys` share the same signed digest for recovery and verification keysets

Severity: Medium

`addKeys(bytes)` and `refreshKeys(bytes)` authenticate the same payload shape using `Codec.keysetDigest(kind, ...)`. The digest is separated by keyset kind, but not by operation mode. The semantic difference between append and replace happens only after signature verification.

Relevant references:

- `contracts/QuipWallet.sol:701`
 - `addKeys(bytes)` calls `_manageKeys(payload, false)`
- `contracts/QuipWallet.sol:706`
 - `refreshKeys(bytes)` calls `_manageKeys(payload, true)`
- `contracts/QuipWallet.sol:1114`
 - `_manageKeys(...)` computes the digest without binding `replace`
- `contracts/QuipWallet.sol:1133`
 - the dangerous semantic split occurs after verification
- `contracts/WOTSPPlusCodec.sol:155`
 - the surrounding comments discuss cross-keyset replay prevention, but do not distinguish append from refresh
- `contracts/WOTSPPlusCodec.sol:1053`

- `keysetDigest(...)` selects a tag by keyset kind only, not by append-vs-refresh intent

Impact:

- a signature intended for append can also validate for replace on the same keyset
- a malicious frontend or transaction builder can flip the selector while preserving the signed payload
- for recovery and verification keysets, “append these keys” can silently become “wipe and replace with these keys”

Recommendation:

- split the digest domains for append vs refresh
- add tests proving `addKeys` signatures do not validate for `refreshKeys` and vice versa



Attack scenario:

1. The owner signs a payload intending to append new recovery or verification keys via `addKeys`.
2. A malicious frontend or relayer submits the same signed payload to `refreshKeys` instead.
3. Signature verification still succeeds because the digest does not distinguish append from replace.
4. The contract clears the existing keyset and replaces it with only the attacker-submitted batch.

4. Upgrade verification signatures are self-authorized rather than independently authorized

Severity: Medium

The upgrade flow appears to require two approvals:

- the wallet's own WOTS+ authorization for performing the upgrade action
- a separate verifier signature over the implementation / upgrade payload

This issue affects both upgrade entrypoints:

- the normal `upgradeToAndCall(...)` path
- the `recoveryUpgrade(...)` path

In both cases, the second approval is not coming from a verifier that was authorized ahead of time. Instead, the payload itself supplies both:

- the `verifier` public key
- the `verifySig` signature that matches that verifier

The contract then checks only that `verifySig` is valid for the calldata-supplied `verifier`. It does not also check that this verifier was previously authorized by wallet state, factory policy, governance, or any stored allowlist.

That makes the second signature a self-declared approval, not an independent approval factor. In other words, the payload is effectively saying "trust verifier X, and here is a valid signature from verifier X," and the contract accepts that as sufficient.

Relevant references:

- `contracts/QuipWallet.sol:872`
 - `verifyUpgrade(...)` decodes `(verifier, verifySig)` directly from calldata
- `contracts/QuipWallet.sol:884`
 - `verifyRecoveryUpgrade(...)` does the same
- `contracts/QuipWallet.sol:1261`
 - `_verifyImplementationSig(...)` checks only that the signature matches the supplied verifier; it does not establish that the verifier was pre-authorized

Impact:

- whoever constructs the payload can choose the supposed verifier keypair themselves

- the “verification signature” is therefore proof of self-consistency, not proof of approval by an independently trusted verifier
- the feature can be mistaken for stronger upgrade governance than it really provides

Recommendation:

- if a real second approval factor is intended, require the verifier to come from a source authorized before the payload is submitted, such as:
 - the wallet’s `verificationKeys` set
 - a factory-managed verifier registry
 - a governance-controlled allowlist
- otherwise document clearly that this is not an independent approval layer



Attack scenario:

1. An attacker controls, or socially engineers, the transaction-key authorization needed for an upgrade.
2. The attacker generates an arbitrary verifier keypair they control.
3. They include that verifier public key and a matching `verifySig` in the upgrade payload.
4. The contract accepts the “second approval” because it checks only self-consistency between the supplied verifier and supplied signature.

5. ERC-1271 replay resistance is operational rather than enforced on-chain

Severity: Low

`isValidSignature(hash, signature)` is view-only and does not consume the verification key. Based on our internal discussions, this is a deliberate design choice driven by the difficulty of safe key rotation in ERC-1271 mode.

Relevant references:

- `contracts/QuipWallet.sol:924`
 - the ERC-1271 path is explicitly documented as stateless and non-consuming
- `contracts/QuipWallet.sol:1004`
 - the digest binds wallet, chain ID, verifier, and `hash`

Impact:

- the same ERC-1271 signature remains valid while the verifier key stays installed
- replay resistance on this surface depends on external rotation discipline, not on-chain enforcement
- integrators can incorrectly assume the same one-time-use guarantees that apply to other WOTS+ paths

Recommendation:

- document prominently that ERC-1271 verification keys are intentionally non-consuming
- add tooling or policy that treats ERC-1271 verification keys as manually burned consumables



Attack scenario:

1. A user produces an ERC-1271 signature for an off-chain approval flow while a verification key is installed.
2. A counterparty or observer stores that signature.
3. The same signature is presented again later while the verifier key is still active.
4. The contract continues to validate it, allowing replay against any integrator that incorrectly assumed one-time semantics.

6. The SDK still targets a stale wallet surface

Severity: Medium

The TypeScript SDK still targets an older wallet interface centered on a single `pqOwner` model and removed method names. This is not limited to a few stale high-level calls: the lower-level TypeScript codec helpers also still encode and decode obsolete payload layouts.

Relevant references:

- `src/index.ts:193`
 - still reads `pqOwner()`
- `src/index.ts:654`
 - still references stale execution surfaces
- `src/wotsCodec.ts:99`
 - `encodeInit(...)` still models the older `pqOwner` centric initialization shape
- `src/wotsCodec.ts:144`
 - `encodeRecoverWallet(...)` still models the older two-key recovery payload shape
- `src/wotsCodec.ts:222`
 - `decodeUpgradeMigration(...)` still reads obsolete upgrade offsets and lengths

Impact:

- SDK consumers can construct malformed payloads or call methods absent from the live ABI
- consumers who bypass `src/index.ts` and use the codec helpers directly are still guided into deterministic payload-shape mismatches
- off-chain integrations can sign or encode payloads that no longer match the live contract surface

Recommendation:

- regenerate the SDK from the current contracts and remove legacy methods
- add parity tests between TS payload builders and Solidity decoders



Attack scenario:

1. An integrator relies on the published SDK to encode wallet actions rather than hand-encoding from the live Solidity contracts.
2. The SDK builds, calls and payloads using stale methods and obsolete codec layouts.
3. Users are induced to sign or submit operations that do not match the live on-chain surface.
4. At best this causes denial of service; at worst it creates signing confusion that a malicious relayer or phishing frontend can exploit.

7. The factory still presents a misleading owner-scoped `vaultId` namespace

Severity: Low

The factory registry is owner-scoped, but deployment salt is globally derived from `vaultId`. That means the external API shape suggests per-owner namespacing, while the actual deployment namespace is global.

Relevant references:

- `contracts/QuipFactory.sol:216`
 - `_deployProxy(...)` uses only `vaultId` as the deployment salt
- `contracts/QuipFactory.sol:243`
 - the registry is still written as `quips[to][vaultId] = contractAddr`

Impact:

- integrators can over-assume that `vaultId` uniqueness is owner-scoped when deployment is actually global
- one user can consume a human-meaningful `vaultId` for all other users

Recommendation:

- document clearly that `vaultId` is globally unique, or derive the salt from both owner and `vaultId`



Attack scenario:

1. A victim chooses a predictable or desirable `vaultId`.
2. An attacker deploys first using that same `vaultId` under their own owner address.
3. Because deployment salt depends only on `vaultId`, the namespace is consumed globally.
4. The victim's later deployment reverts even though the API shape suggests `(owner, vaultId)` scoping.

Operational And Repository Risks

1. The repository mixes multiple contradictory build and release assumptions

Severity: Medium

The codebase currently mixes Foundry and Hardhat pipelines in ways that can produce mismatched artifacts and broken release flows.

Relevant references:

- `foundry.toml:5`
 - `solc 0.8.33`, `evm_version = "cancun"`, optimizer enabled
- `hardhat.config.cts:20`
 - `solc 0.8.28`, `evmVersion = "paris"`, optimizer disabled
- `scripts/release.cts:63`
 - records release metadata assuming the Hardhat compiler settings
- `scripts/copy-abi.js:16`

- exports package ABI/bytecode from Foundry output
- `package.json:39`
 - `release` points to `scripts/release.ts`, but the repo contains `scripts/release.cts`
- validation run in the reviewed environment
 - `npm run build` initially failed because `forge` was not on `PATH`
 - rerunning with Foundry on `PATH` still failed because the Foundry dependency tree was not materialized under `dependencies/`

Impact:

- published artifacts may not represent the same compiler pipeline as documented deployment artifacts
- a clean environment cannot rely on the declared release path
- maintainers can drift into ad hoc release workarounds

Recommendation:

- choose one authoritative release pipeline
- make the release script path and runtime dependencies accurate
- make bootstrap of the Solidity dependency tree explicit and reproducible

2. The SDK has additional safety and usability failures beyond simple stale methods

Severity: Medium

The current SDK does not fail closed on unsupported or changed networks, and it also caches mutable provider state too aggressively:

- `src/addresses.ts:75`
 - `getNetworkAddresses(chainId)` falls back to the default production addresses for any chain ID not explicitly listed
- `src/index.ts:577`
 - `QuipClient` detects the chain once and caches it

- `src/index.ts:581`
 - the factory address is also cached once
- `src/index.ts:572`
 - `QuipClient` also snapshots the connected account once and never refreshes it after account changes
- `src/index.ts:245`
- `src/index.ts:320`
- `src/index.ts:361`
- `src/index.ts:399`
- `src/index.ts:465`
- `src/index.ts:489`
- `src/index.ts:650`
 - all writes pass `chain: null`, disabling viem's chain-consistency guard
- `src/index.ts:801`
 - sequential wallet enumeration treats any read failure as end-of-list

Impact:

- unsupported-chain connections silently fall back to default addresses instead of failing
- a user who switches networks after client construction can still operate against stale chain assumptions
- a user who switches wallet accounts after client construction can still query or transact against vault state for the wrong cached owner
- real RPC failures can appear as "empty wallet list" conditions

Recommendation:

- reject unsupported chains explicitly
- invalidate cached network and account state on provider changes
- keep viem's chain checks enabled unless a stronger local guard replaces them

- only treat specific out-of-bounds conditions as enumeration termination

3. The paymaster is in scope for security, but not in scope for the repo's deployment or packaging flows

Severity: Medium

The report treats `QuipPaymaster` as a first-class trust surface, but the checked-in deployment, address-registry, and packaging paths do not treat it as a first-class shipped component.

Relevant references:

- `src/addresses.ts:29`
 - the exported network address registry includes `Deployer`, `WOTSPPlus`, and `QuipFactory`, but not `QuipPaymaster`
- `script/DeployAll.s.sol:7`
 - the "full deployment" flow only deploys WOTSPPlus and QuipFactory
- `scripts/copy-abi.js:27`
 - ABI export only packages `Deployer`, `QuipFactory`, and `QuipWallet`
- repository search in the reviewed checkout
 - no authoritative deployment script, address export, or packaged ABI path was present for `QuipPaymaster`

Impact:

- operators can assume the paymaster is covered by the same release discipline as the wallet and factory when it is not
- SDK or backend consumers lack one canonical source for paymaster ABI and address material
- security fixes in the paymaster can drift from the repo's documented deployment and release procedures

Recommendation:

- decide explicitly whether the paymaster is a supported shipped component of this repository

- if it is, add it to the authoritative deployment scripts, exported address registry, packaged ABI surface, and release validation flow
 - if it is not, narrow the documented scope and operational claims accordingly
-

4. The package is not self-contained for clean SDK or Forge consumers

Severity: Medium

The current checkout does not bootstrap cleanly from its own checked-in state:

- `package.json:33`
 - `build` shells out to `forge build`
- `package.json:34`
 - `prepare` runs `npm run build`, so even `npm install` attempts a full Solidity build
- `package.json:42`
 - multiple dev dependencies are local `file:../midl-js/...` links
- `src/index.ts:36`
- `src/index.ts:37`
- `src/index.ts:50`
- `src/index.ts:51`
- `src/index.ts:52`
 - the SDK imports generated ABI modules
- validation run in the reviewed environment
 - `./node_modules/.bin/tsc -p tsconfig.build.json --noEmit` failed because `./abi/*.js` had not been generated yet

Impact:

- SDK-only contributors inherit unnecessary coupling to the contract build pipeline
- package installation can fail before consumers reach the actual client surface

- CI and clean-room verification become harder than the repo suggests

Recommendation:

- separate generated ABI artifacts more clearly from handwritten SDK source
 - either commit required generated ABI modules or make generation a mandatory, documented prerequisite
 - reconsider whether `prepare` should force a full Solidity build
-

5. Important safety properties are left as TODOs rather than enforced invariants

Severity: Medium

The repo's TODOs document unresolved safety concerns in the guard and recovery model:

- `TODO.md:5`
 - proposes a stronger final-recourse key because dynamic keyset storage cannot be comprehensively guarded
- `TODO.md:25`
 - proposes a `mustRecover` flag for degenerate key states

Impact:

- maintainers were aware of guard-model and recovery-model gaps without promoting them to enforced invariants
- future work can land on top of partially resolved security assumptions

Recommendation:

- convert security-relevant TODOs into explicit issues with disposition
 - either implement the invariant or document clearly why the current design is still sufficient
-

6. Factory governance and rollout policy remain under-specified

Severity: Medium

The factory is better defended than many comparable systems, but several policy layers remain off-chain conventions rather than enforceable controls:

- no implementation provenance policy beyond codehash equivalence
 - `contracts/QuipFactory.sol:49`
 - `contracts/QuipFactory.sol:89`
- no upgrade-compatibility policy layer
 - `contracts/QuipFactory.sol:49`
 - `contracts/QuipFactory.sol:53`
 - `contracts/QuipFactory.sol:58`
- no staged governance or timelock for sensitive factory actions
 - `contracts/QuipFactory.sol:76`
 - `contracts/QuipFactory.sol:89`
 - `contracts/QuipFactory.sol:106`
 - `contracts/QuipFactory.sol:139`
 - `contracts/QuipFactory.sol:147`
- no distinction between “deployable” and “upgradeable” implementations
- no owner-compatibility policy beyond non-zero-address checks
 - `contracts/QuipFactory.sol:226`

Impact:

- safe rollout and safe upgrade edges depend heavily on operator discipline
- one rushed or compromised owner action can immediately affect implementation policy and fee economics

Recommendation:

- document the intended governance trust model clearly
 - consider explicit compatibility metadata and staged governance for implementation changes
-

7. The assembly is generally deliberate, but a few maintenance hazards remain

Severity: Low

The assembly usage is substantially better defended than average, but several hardening gaps remain:

- `WOTSPPlusCodec` decodes addresses from raw calldata words without explicit masking
 - `contracts/WOTSPPlusCodec.sol:460`
 - `contracts/WOTSPPlusCodec.sol:493`
 - `contracts/WOTSPPlusCodec.sol:593`
- `EnumerableWinternitzAddressSet.values()` relies on a hand-built memory layout
 - `contracts/libraries/EnumerableWinternitzAddressSet.sol:280`
 - `contracts/libraries/EnumerableWinternitzAddressSet.sol:311`
- `QuipPaymaster._verifyAndRotate()` relies on an outer fixed-length precondition to keep its assembly-safe assumptions true
 - `contracts/QuipPaymaster.sol:109`
 - `contracts/QuipPaymaster.sol:294`

Impact:

- these are primarily hardening and maintenance risks, not currently demonstrated exploits
- future refactors can easily break the assumptions that currently make the assembly safe

Recommendation:

- mask decoded addresses explicitly
 - add more comments and regression tests around layout-sensitive assembly paths
-

8. Repository documentation and CI overstate operational maturity

Severity: Low

The docs and CI layout imply more deployment and validation rigor than is actually enforced:

- `README.md:5`
 - the repo admits it is not production-ready, but deployment and release surfaces still look more authoritative than they are
- `MIDL-DEPLOYMENT.md:1`
 - speculative ecosystem notes live alongside authoritative deployment docs
- `ethereum-sdk/.gitlab-ci.yml:1`
 - the checked-in pipeline is still the sample template header
- `ethereum-sdk/.gitlab-ci.yml:26`
 - the only job stages files and creates a git commit; there are no build, test, lint, or release-integrity jobs

Impact:

- operators can mistake speculative guidance for canonical runbooks
- broken release commands, stale generated artifacts, or missing dependencies can merge undetected

Recommendation:

- separate speculative notes from authoritative deployment documentation
- replace the placeholder CI with real build, test, type-check, and release-validation jobs

Testing Gaps

The repository already has better-than-average unit and scenario coverage, but several gaps remain material. The most important missing work is not “more tests”

in the abstract, but more stateful, adversarial, and parity-oriented tests around the exact invariants this system depends on.

1. Missing regression tests for paymaster envelope binding

Current paymaster tests prove that the existing digest accepts the currently intended success path, but they do not mutate excluded sponsorship fields after signing to prove whether validation should fail.

The highest-priority missing negative tests mutate:

- `validUntil`
- `validAfter`
- `paymasterVerificationGasLimit`
- `paymasterPostOpGasLimit`
- `accountGasLimits`
- `preVerificationGas`
- `gasFees`
- `maxCost`

This is the most important test gap because the review found that the digest does not currently bind these fields.

Additional useful regressions in the same area:

- mutate one excluded field at a time after signing and assert whether validation still succeeds
- mutate combinations of excluded fields together, especially gas-limit pairs and validity-window pairs
- replay the same paymaster signature under different `paymasterAndData` headers while holding wallet authorization constant

2. Missing stateful and invariant campaigns for wallet key lifecycle

The repo has strong example-based coverage for individual wallet flows, but it still lacks high-value stateful campaigns over arbitrary valid operation sequences.

The most useful stateful wallet campaigns would generate arbitrary sequences of:

- `execute`
- `withdrawDepositTo`
- `addKeys`
- `refreshKeys`
- `replaceKeyAt`
- `recoverWallet`
- `saveWallet`
- `transferOwnership`
- `completeOwnershipHandover`
- `upgradeToAndCall`
- `recoveryUpgrade`

The most important invariants to encode are:

- no key may exist in more than one wallet key domain at the same time
- no key that has produced a successful authorization should ever become live again later
- transaction-key rotation must always consume the current key before any external interaction becomes reachable
- recovery and disaster-reset paths must not reintroduce historical keys
- ownership reinitialization must leave verification keys cleared and transaction / recovery counts in the expected post-state

This is the single best place to add state-machine testing because many of the project's safety properties emerge only across sequences, not single calls.

3. Missing regression and fuzz tests for `addKeys` vs `refreshKeys` misbinding

The report identified a concrete signature-misbinding issue between append and refresh operations for recovery and verification keysets. That should be locked in with dedicated tests.

Useful additions:

- build a payload signed for `addKeys(Recovery, ...)` and assert whether `refreshKeys(Recovery, ...)` accepts it
 - build a payload signed for `addKeys(Verification, ...)` and assert whether `refreshKeys(Verification, ...)` accepts it
 - if the implementation is fixed, retain those as permanent regressions asserting that cross-mode reuse fails
 - fuzz payloads over varying batch sizes, current keys, next keys, and key contents to ensure append-vs-refresh domain separation holds for all valid shapes
-

4. Missing stateful and invariant campaigns for paymaster verifier lifecycle

The paymaster has a stronger monotonic key model than the wallet, but it still benefits from stateful verification over longer sequences.

The most useful campaigns would cover arbitrary sequences of:

- `setPqVerifier`
- sponsored-operation validation and rotation
- `removePqVerifier`
- attempted re-registration of historical keys
- attempted cross-wallet reuse of prior verifiers

The key invariants to encode are:

- once a verifier hash is marked used, it can never be rebound to any wallet
- a wallet with no registered verifier can never validate sponsorship
- rotating one wallet's verifier can never affect another wallet's verifier state
- successful sponsored operations always advance the stored verifier key

5. Missing higher-level invariant campaigns for factory implementation lifecycle

There is meaningful fuzz coverage for low-level components such as:

- `EnumerableWinternitzAddressSet`
- `WOTSPPlusCodec`

What is still missing are higher-level invariant campaigns for the factory over arbitrary vet / deprecate / undepricate / fee-update sequences.

The most useful invariants to encode are:

- `latestWalletImpl` must always be either zero or a non-deprecated vetted implementation
- a deprecated implementation must never become the selected "latest" implementation without explicit undeprication
- codehash-to-implementation rebinding must preserve whatever provenance policy the project intends
- fee updates must not corrupt implementation-selection state

6. Missing client-surface and parity tests for the published SDK

The TypeScript tests currently cover codec and address helpers, but they still do not cover the highest-risk shipped surface: the actual client API and its interaction assumptions.

The highest-priority additions are:

- ABI existence checks for every method the client calls
- end-to-end tests that instantiate `QuipWalletClient` against a local deployment
- parity tests proving SDK-generated payloads are accepted by the current Solidity contracts
- regression tests for stale method names and removed ABI surfaces
- tests for unsupported-chain handling, cached-chain invalidation, and wallet-enumeration failure modes

- tests asserting that generated ABI modules exist before the SDK build step consumes them

This is the best way to stop the repo from drifting back into a state where the contracts and the shipped SDK describe different systems.

7. Missing targeted tests for upgrade authorization semantics

The upgrade path deserves more explicit tests than it currently has because it combines:

- transaction-key authorization
- implementation vetting
- delegatecall-based verification
- optional migration
- a second verifier path whose intended trust semantics are currently weak

Useful additions:

- regression tests that demonstrate whether arbitrary calldata-supplied upgrade verifiers are accepted
 - negative tests that attempt to use verification keys not meant to be authorized, if the design is tightened
 - stateful tests combining upgrade and migrate flows with subsequent wallet operations to prove key state remains coherent
 - slot-guard regressions using deliberately malicious test implementations
-

8. Missing explicit behavioral tests for ERC-1271 key lifecycle

The current ERC-1271 tests cover direct success and failure branches, but they do not fully lock in the replay model implied by the implementation.

Useful additions:

- repeated `isValidSignature` calls with the same verifier and same message to show that signatures remain valid while the verifier stays installed

- tests proving that replacing or removing the verification key invalidates old signatures
- tests documenting the interaction between owner changes and previously valid ERC-1271 signatures

These are important even if the behavior is intentional, because they make the replay model explicit for integrators.

9. Missing explicit fuzz and invariant configuration in

foundry.toml

The current Foundry config does not define custom `[fuzz]` or `[invariant]` sections, so campaigns depend on Foundry defaults unless manually overridden.

Recommended additions:

```
[fuzz]
runs = 1000

[invariant]
runs = 200
depth = 500
fail_on_revert = true
```

These values are example starting points:

- `fuzz.runs = 1000` chosen as a pragmatic “serious but still usable” baseline for a security-sensitive repo
- `invariant.runs = 200` chosen to give repeated state-machine exploration without making every run excessively heavy
- `invariant.depth = 500` chosen because the wallet, paymaster, and factory all require multi-step sequence coverage rather than single-call fuzzing
- `invariant.fail_on_revert = true` so unexpected reverts surface as failures instead of being silently tolerated

They should be treated as CI-friendly defaults to tune later, not as project-specific optimum values.

Overall Assessment

Aspect	Rating
Smart contract architecture	
Core security design	
Test coverage	
Documentation / operational clarity	
Build / release hygiene	
Deployment / packaging completeness	

The Solidity side shows a serious amount of defensive engineering, especially around:

- replay prevention
- WOTS+ key hygiene
- guarded delegatecall verification
- cross-keyset uniqueness
- reentrancy-aware rotation ordering

The test suite is also materially stronger than average for an early-stage wallet/paymaster system.

The main weaknesses are not “basic solidity mistakes.” They cluster around:

- incomplete binding between signatures and full operation semantics
- recovery and governance semantics that are weaker than their naming suggests
- SDK and packaging drift relative to the live contracts
- build, release, and CI assumptions that are not yet reproducible from the checked-in repository alone

The project is thoughtful and technically serious, but not yet in the shape where the repository itself can be treated as a clean, authoritative, one-command-validatable release surface. The next highest-value work is:

1. fix the paymaster envelope binding
2. enforce monotonic wallet key burn
3. split append-vs-refresh key-management digest domains
4. bring the SDK fully back into sync with the contracts
5. make build, release, and CI paths reproducible from a clean checkout